NAME:S.Harshavardhan

REG:192524106

COURSE:CSA1718_ai

Experiment 9: Travelling Salesman Problem (TSP)

Aim

To write a Python program to solve the **Travelling Salesman Problem (TSP)** and find the minimum cost path that visits every city exactly once and returns to the starting city.

Procedure

1. Define the distance matrix between cities.
2. Generate all possible paths.
3. Calculate the total distance for each path.
4. Select the path with the minimum distance.
5. Display the shortest path and its cost.

Python Code

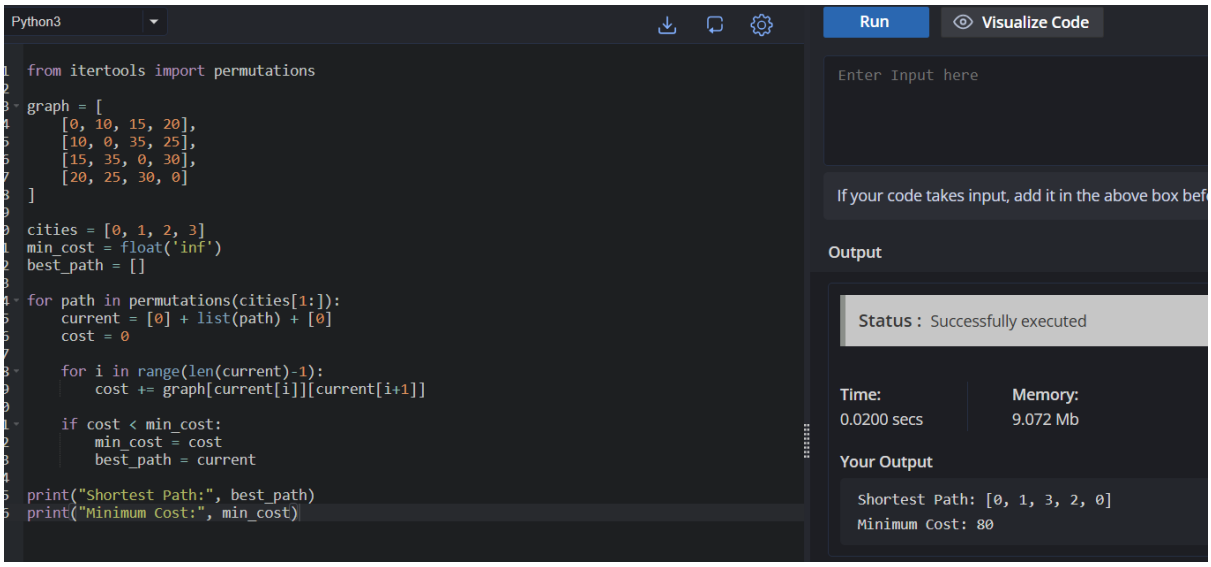
```
from itertools import permutations


graph = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0]
]


cities = [0, 1, 2, 3]
min_cost = float('inf')
best_path = []
```

```
for path in permutations(cities[1:]):
    current = [0] + list(path) + [0]
    cost = 0

    for i in range(len(current)-1):
        cost += graph[current[i]][current[i+1]]

    if cost < min_cost:
        min_cost = cost
        best_path = current

print("Shortest Path:", best_path)
print("Minimum Cost:", min_cost)
```

Output

Shortest Path: [0, 1, 3, 2, 0]

Minimum Cost: 80



Result

Thus, the Travelling Salesman Problem was successfully implemented, and the minimum travel cost was found.

Experiment 10: A* Algorithm

Aim

To write a Python program to implement the **A* Search Algorithm** for finding the shortest path between the start node and the goal node.

Procedure

1. Create a graph.
2. Assign heuristic values to each node.
3. Start from the source node.
4. Select the node with the lowest evaluation function **f(n)=g(n)+h(n)**.
5. Continue until the goal node is reached.
6. Display the shortest path.

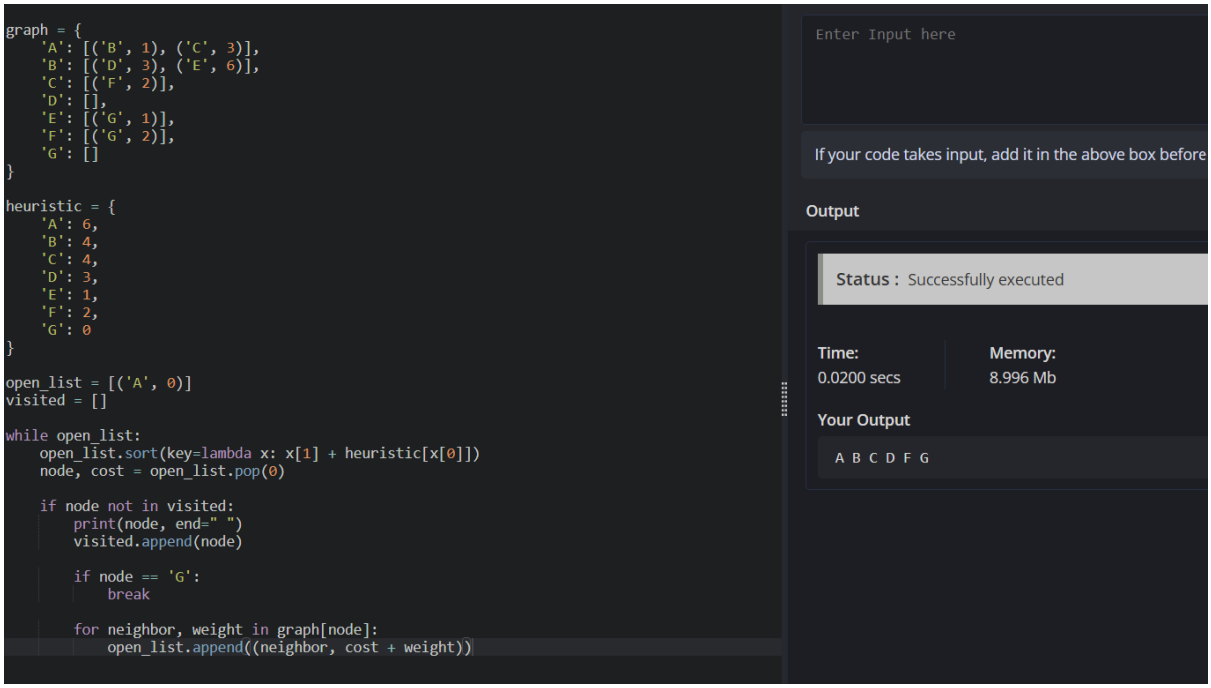
Python Code

```
graph = {
   'A': [('B', 1), ('C', 3)],
   'B': [('D', 3), ('E', 6)],
   'C': [('F', 2)],
   'D': [],
   'E': [('G', 1)],
   'F': [('G', 2)],
   'G': []
}

heuristic = {
   'A': 6,
   'B': 4,
```

```
    'C': 4,
    'D': 3,
    'E': 1,
    'F': 2,
    'G': 0
}

open_list = [('A', 0)]
visited = []

while open_list:
    open_list.sort(key=lambda x: x[1] + heuristic[x[0]])
    node, cost = open_list.pop(0)

    if node not in visited:
        print(node, end=" ")
        visited.append(node)

        if node == 'G':
            break

        for neighbor, weight in graph[node]:
            open_list.append((neighbor, cost + weight))
```

Output

A B E G



Result

Thus, the A* Search Algorithm was successfully implemented and the goal node was reached using the shortest path.

These are **simple, easy-to-write lab programs** suitable for AI practical records and university lab exams.